



University of Camerino

SCIENCE AND TECHNOLOGIES

Master Degree in Computer Science (LM-18 Class)
Fundamentals of System Verification

Modelling and Verification of a Ring-Based Leader Election Protocol

Students
Marvin Sincini
Student ID 130207

Unicam Supervisors
Prof. Luca Tesei

A.Y. 2025/2026

Contents

1	Introduction	5
2	Protocol Description	7
3	Modelling in PROMELA	9
4	Progressive Refinement of the PROMELA	11
5	Model Comparison between the naive and the refined PROMELA models	13
6	Modelling in UPPAAL	15
7	Abstraction Choices	17
8	Safety Properties	19
9	Liveness Properties	21
10	Experimental Evaluation	23
11	Conclusions	25

1. Introduction

Distributed protocols are often difficult to analyze using purely informal reasoning, because their behaviour depends on the interaction among several concurrent processes and on the order in which messages are delivered. Therefore, even conceptually simple protocols may exhibit behaviours that are not immediately evident, especially when communication is asynchronous.

This project considers a leader election protocol in a unidirectional ring network. Each process has a unique identifier and communicates only with its successor. The goal of the protocol is to elect as leader the process with the maximum identifier. The mechanism is based on the circulation of identifiers: each node forwards only messages containing identifiers greater than its own, discards smaller ones, and elects itself as leader when it receives its own identifier again.

The aim of the work is not only to model the protocol, but also to formally verify some fundamental properties. In particular, the verification considers safety properties, such as the impossibility of electing more than one leader or electing a node different from the one with the maximum identifier, and liveness properties, such as the fact that a leader is eventually elected.

The protocol has been modelled using two different tools. First, a PROMELA model was developed and verified with SPIN. Then, the protocol was modelled in UPPAAL as a network of finite-state automata, explicitly introducing bounded FIFO buffers to represent asynchronous communication.

Finally, the project includes an experimental evaluation of the impact of the number of processes and of the buffer capacity on the state space. This analysis makes it possible to observe the state explosion problem concretely and to compare whether different modelling choices affect the verifiability of the system.

2. Protocol Description

The protocol considered in this project is a distributed leader election protocol over a unidirectional ring network. The system is composed of a finite set of processes, each associated with a unique identifier. The processes are logically arranged in a ring: each process can send messages only to its successor, while it receives messages only from its predecessor.

This topology provides some useful properties. In particular, every process can reach every other process after a finite number of steps, and a process can receive its own message again only after that message has traversed the whole ring.

Communication among processes is asynchronous, so messages are inserted into communication channels with bounded capacity. Each message contains a single value, namely the identifier of a process.

At the beginning of the execution, each process sends its own identifier to its successor. Then, each process waits to receive a message from its predecessor and compares the received identifier with its local identifier. If the received identifier is smaller than its own, the message is discarded, because that candidate cannot become leader anymore. If the received identifier is greater than its own, the process forwards the message to its successor, since there exists a candidate in the ring with higher priority. Finally, if a process receives a message containing its own identifier, this means that the identifier has traversed the entire ring without being discarded. In this case, the process elects itself as leader.

The correctness of the protocol depends on the fact that all identifiers are distinct. Indeed, an identifier can complete a full traversal of the ring only if it never encounters a process with a greater identifier. Consequently, the only identifier that can return to its originating process is the maximum one. All other identifiers are necessarily discarded by some process with a higher identifier.

In the model used in this project, the process with the maximum identifier is therefore the only candidate that can be elected leader. The protocol logically terminates when this process receives its own identifier again and sets its leader state. Intuitively, only the process with the highest identifier can become leader, because every other identifier must eventually pass through a process with a greater identifier and will therefore be discarded.

3. Modelling in PROMELA

The protocol was modeled in PROMELA by representing each node of the ring as a concurrent process. The number of processes is defined by the constant `N`, while the capacity of the asynchronous channels is defined by the constant `CAP`. In this way, it is possible to easily modify the size of the system and observe the impact of these parameters on the verification.

Communication among nodes is modeled through an array of PROMELA channels:

```
chan ch[N] = [CAP] of { byte };
```

Each channel represents the input buffer of a process. In particular, process `i` receives messages from channel `ch[i]` and sends its own messages to the channel of its successor, namely `ch[(i + 1) % N]`. The modulo operator is used to close the ring structure, so that the last process communicates again with the first one.

The identifiers of the processes are stored in the array `id`, while the Boolean array `leader` indicates which processes have been elected as leaders. Moreover, the global variable `leader_count` keeps track of the total number of elected leaders. This variable is useful both for expressing correctness properties, such as the absence of multiple leaders, and for detecting when the election has occurred.

In the final model used for the main verification, seven processes are considered. The identifiers are assigned deterministically during initialization, according to a non-ordered arrangement. For `N = 7`, the resulting sequence is:

1, 7, 2, 6, 3, 5, 4

Consequently, the process with index 1 has the maximum identifier and is the only process that should be able to become leader. The identifier generation algorithm allows the number of processes to vary, keeps the identifiers in a scattered order, and, most importantly, keeps the node that is expected to become leader constantly in position 1.

The behavior of each node is described by the PROMELA process `Node`. At startup, each process computes its successor and sends its own identifier to the input channel of that successor. After this initial phase, the process enters a loop in which it receives messages from its input channel and compares them with its local identifier.

If the received message contains an identifier greater than its own, the process forwards it to its successor. If the message contains a smaller identifier, it is discarded. Finally, if the message contains an identifier equal to the process identifier, the process elects itself as leader, updates its value in the `leader` array, and increments `leader_count`.

The general structure of the node behavior is therefore the following:

```
ch[next]!id[i];
```

```
do
```

```
:: ch[i]?msg ->
```

```
  if
```

```
    :: msg > id[i] ->  
      ch[next]!msg
```

```
    :: msg < id[i] ->  
      skip
```

```
    :: msg == id[i] ->  
      leader_count = leader_count + 1;  
      leader[i] = true;
```

```
  fi
```

```
od
```

The `init` process is responsible for initializing the identifiers, setting all values of the `leader` array to `false`, and creating the concurrent processes through the `run` instruction. In this way, all nodes of the ring are activated and can execute the protocol independently, producing the possible interleavings analyzed by SPIN.

Some LTL properties were also embedded directly in the PROMELA model. These properties concern both safety aspects, such as the fact that more than one leader cannot be elected and that only the process with the maximum identifier can become leader, and liveness aspects, such as the eventual election of a leader. These properties will be discussed in more detail in the sections dedicated to safety and liveness.

4. Progressive Refinement of the PROMELA

The first version of the model directly represents the behavior of the protocol: each process initially sends its own identifier, then receives messages from its input channel and decides whether to discard them, forward them, or elect itself as leader. This version is useful because it describes the main logic of the protocol in a simple and readable way.

However, the naive model presents a termination-related issue. After the leader is elected, the other processes still remain inside the receiving loop, waiting for new messages. From the point of view of the protocol, the election has already occurred correctly; from the point of view of SPIN, however, the system may reach a configuration in which all processes are blocked in states that are not explicitly final. For this reason, the verification may report an invalid end state, even though the main safety properties have not been violated.

The refined version of the model therefore introduces an explicit termination after the election of the leader. Each process checks the global variable `leader_count`: when this value becomes greater than zero, a leader has been elected and the process can exit the main loop, reaching the final state `end_state`.

```
do
  :: leader_count > 0 ->
    goto end_state

  :: atomic {
    ...
  }
od

end_state: skip
```

This modification makes it possible to clearly distinguish between a real error situation and the logical conclusion of the protocol. After the election, the processes no longer need to wait for new messages; instead, they can terminate correctly.

A second refinement concerns the management of asynchronous communication. In the naive model, a process receives a message from its input channel and only afterwards tries to forward it to its successor, if necessary. This modeling choice is simple, but if a process removes a message from its own channel and then cannot forward it because the successor channel is full, the message has already been consumed from the input channel before the forwarding action is guaranteed to be executable.

To represent the receive-and-forward operation in a more controlled way, the refined

version first uses a non-destructive read of the message through `ch[i]?<msg>`.

```
atomic {  
    ch[i]?<msg>;  
  
    if  
    :: leader_count > 0 ->  
        goto end_state  
  
    :: msg > id[i] && nfull(ch[next]) ->  
        ch[i]?msg;  
        ch[next]!msg  
  
    :: msg < id[i] ->  
        ch[i]?msg  
  
    :: msg == id[i] ->  
        ch[i]?msg;  
        leader[i] = true;  
        leader_count = leader_count + 1  
    fi  
}
```

The non-destructive read allows the process to observe the message at the head of the channel without immediately removing it. The message is actually consumed only when the following action is executable. In the forwarding case, the guard `nfull(ch[next])` checks that the successor channel is not full. Only in that case does the process remove the message from its own channel and send it to the successor.

The use of the `atomic` block also makes the sequence composed of message inspection, decision, and possible channel update indivisible. This reduces non-significant interleavings between operations that, at the abstract level, represent a single reaction of the node to the reception of a message.

5. Model Comparison between the naive and the refined PROMELA models

The naive model and the refined model differ both in the level of precision with which they describe termination and in the way they model asynchronous communication.

In the naive model, each node remains indefinitely inside the receiving loop. This means that, even after the leader has been elected, the processes continue to wait for possible new messages. From the abstract point of view of the protocol, this is not necessarily an error: the main goal, namely electing the process with the maximum identifier, has already been achieved. However, from the point of view of SPIN, the system may terminate in a configuration in which the processes are blocked on receive operations that are no longer executable. Since these states are not marked as final states, the verifier may report them as invalid end states.

In the refined model, instead, termination after the election is modeled explicitly. When the global variable `leader_count` becomes greater than zero, each process can exit the main loop and reach the `end_state` state. In this way, a configuration in which the election has been completed is no longer interpreted as an abnormal termination, but as the correct conclusion of the protocol.

A second difference concerns message management. In the naive model, reception from the input channel occurs immediately through `ch[i]?msg`. After this reception, if the message must be forwarded, the process tries to send it to its successor. This solution is compact, but it does not clearly distinguish between observing the message and actually removing it from the channel.

In the refined model, instead, the process first uses a non-destructive read, `ch[i]?<msg>`, to inspect the message at the head of the channel. The message is then removed only when the following action is actually possible. In the forwarding case, this also requires the successor channel not to be full, a condition checked through `nfull(ch[next])`.

Therefore, the refined model is the only one suitable for the final verification, because it eliminates the invalid end states caused by the lack of explicit termination and represents receive and forward operations more precisely in the presence of bounded asynchronous channels.

6. Modelling in UPPAAL

The protocol was also modeled in UPPAAL by representing each process of the ring as an instance of the same parametric template, called `Node`. The template parameter is the process index, denoted by `i`, which allows each instance to access its own identifier, its input buffer, and the logical channel of its successor.

Unlike PROMELA, in UPPAAL the structure of the system is mainly represented through finite-state automata. For this reason, the behavior of each node was divided into several locations, each corresponding to a logical phase of the protocol. The main locations of the template are `Init`, `Waiting`, `Read`, `Send`, `Found`, and `Done`.

The location `Init` represents the initial phase of the process. In this phase, the node sends its own identifier to its successor. After the initial send, the process moves to `Waiting`, where it waits until a message is present in its input buffer. When a message is available, the process moves to `Read`, where it reads the value at the head of the buffer and stores it in the local variable `msg`.

The location `Read` represents the decision point of the protocol. If the read message contains an identifier smaller than the identifier of the current process, the message is discarded and the process returns to `Waiting`. If the message contains a greater identifier, the process moves to the `Send` location, from which it forwards the message to its successor as soon as the destination buffer is available. Finally, if the message contains an identifier equal to the identifier of the current process, the process elects itself as leader.

The location `Send` was introduced to separate the decision to forward a message from the actual send operation. This separation makes explicit the fact that forwarding can occur only if the successor buffer is not full. In this way, the model preserves the asynchronous and bounded nature of communication.

The location `Found` represents an intermediate phase reached when a leader has already been elected. In this case, the process no longer has to continue the normal receive-and-forward cycle, but can move toward the final state. The location `Done` therefore represents the logical final state of the process after the conclusion of the election. A self-loop is present on `Done`, in order to avoid artificial deadlocks after the protocol has completed its useful activity.

Asynchronous communication was modeled explicitly through bounded FIFO buffers. Each process has an input buffer, represented by the global array `buffer`, and a variable `size`, which indicates the number of messages currently stored in the buffer.

The function `push` inserts a message into the destination buffer, while `pop` removes the message at the head of the buffer and shifts any following messages forward. The function `front` reads the message at the head of the buffer without immediately removing it. The Boolean functions `can_send` and `can_receive` are used as guards on the automaton edges, respectively to check that a buffer is not full and that a buffer is

not empty.

The main configuration of the UPPAAL model uses three processes, with identifiers 1, 3, and 2. Therefore, the process with index 1 has the maximum identifier and is the candidate that must be elected leader. The template instances are declared in the system as P0, P1, and P2, all created from the same Node template.

Some global variables useful for verification were also introduced in the model. The Boolean array `leader` indicates which processes have been elected as leaders, while `leader_count` counts the total number of elected leaders. The variable `wrong_leader` is set to `true` if a process different from the one with the maximum identifier tries to elect itself as leader. These variables make it possible to express the correctness properties verified later in UPPAAL in a simple way.

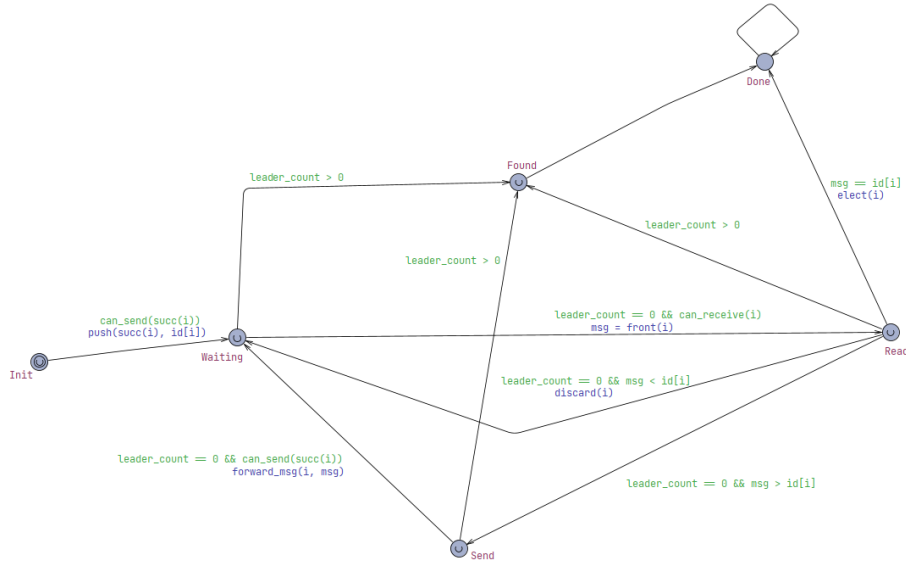


Figure 6.1: UPPAAL template of a ring node.

7. Abstraction Choices

During the modeling phase, some abstractions were introduced in order to keep the system finite, verifiable, and coherent with the goals of the project. The objective was not to simulate every possible detail of a real distributed network, but to faithfully represent the essential aspects of the leader election protocol.

The first abstraction concerns the size of the system. The number of processes is fixed through the constant `N`, so that the model remains finite. Both in PROMELA and in UPPAAL, this parameter was varied during the experimental evaluation in order to observe the effect of the number of nodes on the state space.

Communication is modeled as asynchronous and based on bounded buffers. However, transmission errors, spontaneous message losses, message duplication, and process failures are not considered. Messages may be delayed because of action interleavings and limited channel capacity, but they are not altered by the network. This abstraction is coherent with the goal of the verification, which is to study the correctness of the protocol in the presence of concurrency and asynchronous communication, not its fault tolerance.

The message format was also simplified. Each message contains only the identifier of a process. No additional fields, different message types, timestamps, or information about the followed path are present. This choice is natural for the considered protocol, since each node's decision depends exclusively on the comparison between its own identifier and the received one.

Another choice concerns the termination of the protocol. In the abstract behavior of the protocol, the objective is achieved when the process with the maximum identifier receives its own identifier again and elects itself as leader. A subsequent global announcement phase of the completed election is not modeled. Also to avoid ambiguous final states, explicit process termination after the election was introduced in both the refined PROMELA model and the UPPAAL model.

In the refined PROMELA model, some sequences of operations are enclosed in an `atomic` block. This choice abstracts the local reaction of a node to the reception of a message as a single logical action. In this way, non-significant interleavings are reduced without changing the logic of the protocol.

In the UPPAAL model, no clocks were used, since the protocol does not depend on timing constraints. Moreover, since UPPAAL does not directly provide the same mechanism of asynchronous FIFO channels available in PROMELA, the buffers were modeled explicitly through arrays and size variables.

Finally, some global variables, such as `leader_count` and `wrong_leader`, were introduced mainly to simplify the expression of the properties to be verified. These variables are not necessarily part of a real implementation of the protocol, but they act as observational variables in the model. They make it possible to formulate properties

such as ‘at most one leader” or ‘no process different from the one with the maximum identifier is elected” in a simple way.

8. Safety Properties

Safety properties describe conditions that must never be violated during the execution of the protocol. In the case of the leader election protocol, the main safety properties concern the impossibility of electing more than one leader, the impossibility of electing a process different from the one with the maximum identifier, and, in the refined model, the absence of undesired deadlocks.

The first property requires that, in every reachable configuration, the number of elected leaders is at most one. In the PROMELA model, this property is expressed through the global variable `leader_count`, which is incremented every time a process elects itself as leader.

```
ltl at_most_one {  
    [] (leader_count <= 1)  
}
```

This formula states that, along every execution, `leader_count` must never exceed the value 1. If this property were violated, it would mean that at least two different processes managed to elect themselves as leaders, making the protocol incorrect.

The second safety property requires that only the process with the maximum identifier can become leader. In the main configuration with $N = 7$, the sequence of identifiers is 1, 7, 2, 6, 3, 5, 4; therefore, the process with index 1 is the only process that should be able to become leader. In PROMELA, this property was expressed by requiring that all the other processes never become leaders.

```
ltl only_max_can_lead {  
    [] (!leader[0] && !leader[2] && !leader[3] &&  
        !leader[4] && !leader[5] && !leader[6])  
}
```

This formula does not directly require the maximum process to be elected, but it guarantees that no process different from it can be elected. The actual election of the maximum process is instead verified through a liveness property.

In the refined model, the absence of undesired deadlocks was also checked using SPIN.

Equivalent properties were verified in UPPAAL using verifier queries. The property that prevents the election of more than one leader is expressed as:

```
A[] leader_count <= 1
```

This query states that, in all reachable states, the number of elected leaders is at most one. The property that prevents the election of an incorrect leader is instead expressed through the observational variable `wrong_leader`:

```
A[] !wrong_leader
```

In the model, `wrong_leader` is set to `true` if a process different from the one with the maximum identifier tries to elect itself as leader. Therefore, the query verifies that this situation is never reachable.

Finally, the absence of deadlocks was also verified in UPPAAL:

`A[] not deadlock`

This property ensures that the model does not reach unexpected blocked configurations. In the UPPAAL model, after the election of the leader, the processes can reach the `Done` location, which has a self-loop. In this way, the logical termination of the protocol is not treated as an artificial deadlock.

9. Liveness Properties

Liveness properties describe conditions that must eventually become true during the execution of the protocol. In the case of the leader election protocol, the main liveness properties concern the eventual election of a leader and, more specifically, the eventual election of the process with the maximum identifier.

The first property requires that, along every execution, at least one leader is eventually elected. In the PROMELA model, this property was expressed by checking that the global variable `leader_count` eventually becomes greater than zero.

```
ltl eventually_some_leader {  
    ◇ (leader_count > 0)  
}
```

This formula states that there must not exist an infinite execution in which no process is ever elected as leader. If this property were violated, the protocol could continue indefinitely without reaching its main objective.

The second liveness property requires that the process with the maximum identifier is actually elected. In the main configuration with $N = 7$, the process with index 1 has the maximum identifier. For this reason, the property was expressed by checking that `leader[1]` eventually becomes true.

```
ltl eventually_max {  
    ◇ leader[1]  
}
```

In the PROMELA model, a property related to the persistence of the leader state was also considered. Once the maximum process has been elected, it must remain leader for the rest of the execution.

```
ltl leader_persistence {  
    [] (leader[1] -> [] leader[1])  
}
```

This formula states that, at every point of the execution, if `leader[1]` becomes true, then from that moment on it will always remain true. In the model, this property is also guaranteed by the structure of the code, since no transition assigns the value false again to any position of the `leader` array.

Equivalent liveness properties were verified in UPPAAL through verifier queries. The eventual election of a leader is expressed by the following query:

```
A◇ leader_count > 0
```

This query requires that, along all paths of the system, a state in which at least one leader has been elected is eventually reached.

The eventual election of the correct process is instead expressed as:

$A \Diamond \text{leader}[\text{MAX_PID}]$

The constant `MAX_PID` identifies the process with the maximum identifier. Therefore, the query verifies that, in all executions considered by the model, the correct process is eventually elected as leader.

The persistence property was not expressed in UPPAAL through a nested temporal formula, because the UPPAAL query language does not directly allow properties of the form “if something happens, then from that moment on it always remains true”. However, persistence is also guaranteed by construction in the UPPAAL model: the variables of the `leader` array are initially false, can be set to true during the election, and no transition sets them back to false.

10. Experimental Evaluation

The experimental evaluation was carried out to observe the impact of the number of processes and of the buffer capacity on the size of the state space. The main experiments were performed on the refined PROMELA model, since this is the model suitable for the final verification: it does not present invalid end states caused by the lack of explicit termination, and it handles message reception and forwarding more precisely.

For each configuration, two parameters were varied: the number of processes N and the channel capacity CAP . In particular, values of N between 6 and 13 were considered, while CAP was assigned the values 1, 2, and 3. For each combination, the number of stored states, matched states, transitions, maximum depth reached, memory usage, and verification time were collected.

N	CAP	state_vector	stored	matched	transitions	depth	errors	memory_mb	time_s
6	1	144	1085	1567	2652	267	0	192.39.00	0.00
6	2	144	942	1281	2223	265	0	192.39.00	0.00
6	3	144	901	1221	2122	264	0	192.39.00	0.01
7	1	160	2979	5922	8901	324	0	195.54.00	0.00
7	2	160	2693	5229	7922	320	0	195.54.00	0.01
7	3	160	2423	4611	7034	319	0	195.54.00	0.01
8	1	184	7952	20108	28060	373	0	190.00.00	0.02
8	2	184	6535	15686	22221	369	0	188.22.00	0.01
8	3	184	5912	14159	20071	368	0	188.22.00	0.01
9	1	200	23412	72581	95993	436	0	196.06.00	0.02
9	2	200	20629	62312	82941	430	0	192.51.00	0.03
9	3	200	17322	51741	69063	428	0	186.20.00	0.02
10	1	216	65122	238018	303140	490	0	206.16.00	0.07
10	2	216	51931	181506	233437	484	0	195.54.00	0.06
10	3	216	43766	153446	197212	482	0	201.48.00	0.06
11	1	240	198215	836874	1035089	559	0	223.57.00	0.30
11	2	240	173387	711430	884817	550	0	220.05.00	0.25
11	3	240	134793	552323	687116	547	0	207.16.00	0.20
12	1	256	562991	2685919	3248910	618	0	276.35.00	1.35
12	2	256	445766	2039801	2485567	609	0	271.06.00	1.11
12	3	256	347366	1605802	1953168	606	0	257.17.00	1.04
13	1	272	1744135	9277086	11021221	693	0	496.18.00	3.45
13	2	272	1539625	7924231	9463856	681	0	486.45.00	3.31
13	3	272	1105643	5731036	6836679	677	0	414.00.00	2.18

Figure 10.1: Experimental results for the refined PROMELA model.

All the considered configurations were verified with 0 errors. This confirms that, for the analyzed values, the refined model satisfies the checked properties and does not produce invalid end states or undesired deadlocks.

The most evident result concerns the growth of the state space as the number of processes increases. For example, with $CAP = 1$, the number of stored states grows from 1085 for $N = 6$ to 1744135 for $N = 13$. The number of transitions also increases significantly, from 2652 to 11021221. This trend clearly shows the state explosion problem: even a simple protocol, when modeled as a concurrent system with several processes, can generate a very large number of reachable configurations.

A second observable aspect concerns the effect of buffer capacity. In the obtained results, increasing CAP from 1 to 3 often decreases the number of states and transitions. This behavior depends on the specific structure of the refined model: smaller buffers introduce more temporary blocking situations and therefore more interleavings related to waiting for channel availability. With slightly larger buffers, some of these situations are reduced, and the resulting state space can become more compact.

Memory usage and verification time grow consistently with the size of the state space. The smallest configurations are verified almost immediately, while the largest considered configuration, with $N = 13$ and $CAP = 1$, requires more memory and a longer verification time. Nevertheless, all tested configurations remain verifiable.

The UPPAAL model was also verified using the queries defined in the previous sections. In particular, the checked properties include the absence of deadlocks, the impossibility of electing more than one leader, the impossibility of electing an incorrect leader, the eventual election of a leader, and the eventual election of the process with the maximum identifier. All queries are satisfied, confirming the correctness of the represented behavior also in the automata-based model.

Additional UPPAAL versions with a greater number of nodes and larger buffer sizes were also tested. The obtained results show that, under these parameter variations, the system remains robust and consistent: the same correctness properties continue to be satisfied.

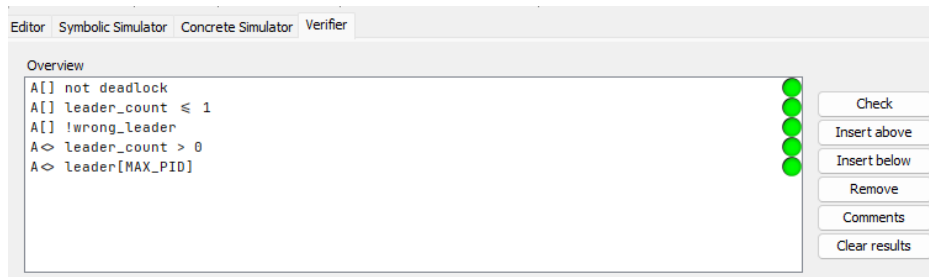


Figure 10.2: UPPAAL verification results.

11. Conclusions

The PROMELA modeling made it possible to directly represent asynchronous communication among processes through channels with bounded capacity. The first version of the model, the naive one, highlighted a typical issue in formal verification: even when the protocol correctly reaches the election of the leader, the absence of explicit termination may lead SPIN to report invalid end states. For this reason, a refined version was introduced, in which the processes logically terminate after the election and message handling is made more precise through non-destructive reads, checks on the availability of the successor channel, and atomic blocks.

The UPPAAL modeling provided an alternative representation of the same protocol, based on finite-state automata. In this case, since UPPAAL does not directly provide asynchronous FIFO channels equivalent to those of PROMELA, the buffers were modeled explicitly through arrays and size variables. This choice made it possible to preserve the asynchronous and bounded nature of communication also within a different formalism.

The verified properties confirm the correctness of the protocol with respect to the considered objectives. The safety properties show that more than one leader cannot be elected, that no process different from the one with the maximum identifier can become leader, and that the refined model does not present undesired deadlocks. The liveness properties show instead that the election is actually reached and that the elected process is the correct one. Moreover, the persistence of the leader state is guaranteed by the structure of the models, since no transition sets a leader variable back to false after it has become true.

The experimental evaluation highlighted the impact of the number of processes and of the buffer capacity on the size of the state space. In particular, increasing the number of processes produces a significant growth in the number of reachable states and transitions, making the state explosion problem evident. At the same time, all analyzed configurations were verified with zero errors, confirming the correctness of the refined model in the considered cases.

Overall, the combined use of SPIN/PROMELA and UPPAAL made it possible to analyze the same protocol from two complementary perspectives: asynchronous communication through channels on one side, and automata-based representation on the other. Under the adopted abstractions, the protocol satisfies the required properties and correctly elects a single leader.